Taller de Desarrollo de Software

SENA CNCA – Nodo Tic ADSO17

🎬 Sistema Web para Gestión de Cine

Nombre de la actividad:

Desarrollo de una Aplicación Web para la Gestión de Cartelera y Venta de Tiquetes de Cine

🎯 Objetivo de la actividad

Desarrollar una aplicación web que permita sistematizar el proceso de publicación de películas, gestión de funciones, venta de tiquetes y control de acceso a una sala de cine, aplicando buenas prácticas de desarrollo de software, trabajo colaborativo y uso de tecnologías web.

🧠 Contexto del problema

Una empresa de cine necesita digitalizar sus procesos, ya que actualmente presenta dificultades en:

- La publicación manual de su cartelera.
- La organización de funciones.
- La venta de tiquetes sin control en tiempo real.
- La validación de los tiquetes en el ingreso a la sala.

La sala cuenta con una capacidad de **150 sillas**, y es necesario evitar la sobreventa y garantizar un control eficiente de los asistentes.

👥 Modalidad de trabajo

- Trabajo en equipos de **máximo 4 aprendices**.
- Cada equipo deberá organizar roles (ejemplo):
  - Líder de equipo
  - Desarrollador Backend
  - Desarrollador Frontend
  - QA / Documentador

⏳ Duración

2 semanas (10 a 12 días hábiles)

🧩 Requerimientos del sistema

1. Gestión de películas

- Registrar, editar, eliminar y listar películas.
- Datos mínimos: título, género, duración, clasificación, descripción.

2. Programación de funciones

- Crear funciones por película (fecha y hora).
- Asociar funciones a una sala con **150 sillas disponibles**.

3. Gestión de asientos

- Visualización gráfica de las sillas (tipo grid).
- Selección de asientos disponibles.
- Bloqueo de asientos ocupados.

4. Venta de tiquetes

- Selección de película, función y asientos.
- Cálculo del valor total.
- Generación de tiquete con código único.

5. Validación de tiquetes

- Ingreso o escaneo del código del tiquete.

- Validación del estado:
    - Válido
    - Usado
    - Inválido

6. Panel administrativo (básico)

- Visualización de:
    - Total de ventas
    - Ocupación de sala

⚙️ Requerimientos técnicos

- Uso de tecnologías web:
    - HTML, CSS, JavaScript
    - Framework opcional (React, Vue, Bootstrap)
- Backend:
    - Node.js, Flask o Laravel
- Base de datos:
    - MySQL o PostgreSQL
- Uso de API para comunicación cliente-servidor.
- Implementación de validaciones.

📦 Entregables

Cada equipo deberá entregar:

1. Código fuente del proyecto.
2. Base de datos (script SQL).
3. Documento técnico que incluya:
    - Descripción del sistema
    - Arquitectura utilizada

    - Modelo de base de datos (diagrama ER)
    - Endpoints del API
4. Video demostrativo (máx. 5 minutos).
5. Aplicación funcional desplegada en la nube.

🗓️ Cronograma sugerido

Día	Actividad
1 - 2	Análisis del problema y definición de requerimientos
3 - 4	Diseño (mockups y base de datos)
5 - 8	Desarrollo (frontend y backend)
9	Pruebas y correcciones
10	Presentación y entrega final

📊 Criterios de evaluación

Criterio	**Porcentaje**
Funcionamiento del sistema	30%
Estructura y calidad del código	20%
Diseño de interfaz (UX/UI)	15%
Implementación de base de datos	15%
Documentación	10%
Trabajo en equipo y presentación	10%

🚀 Resultados de aprendizaje esperados

Al finalizar la actividad, el aprendiz estará en capacidad de:

- Analizar un problema real y proponer una solución tecnológica.
- Diseñar y desarrollar una aplicación web funcional.
- Implementar bases de datos relacionales.
- Trabajar colaborativamente en proyectos de software.
- Aplicar buenas prácticas de desarrollo.

💡 Recomendaciones

- Utilizar control de versiones (Git).
- Dividir tareas claramente entre los integrantes.
- Priorizar funcionalidades principales antes de agregar extras.
- Probar continuamente el sistema.

A continuación, te comparto la estructura, como lo haría un arquitecto de software, pensando en un sistema escalable, claro para los aprendices y fácil de implementar en MySQL/PostgreSQL.

🗄️ **1. Estructura de Base de Datos**

🔹 **Entidades principales**

👤 **usuarios (opcional pero recomendado)**

Para clientes y administradores.

- id (PK)
- nombre
- email
- contraseña
- rol (admin / cliente)
- fecha_creacion

🎬 **peliculas**

- id (PK)
- titulo
- descripcion
- duracion (minutos)
- genero
- clasificacion (Ej: +13, +18)
- imagen_url
- trailer_url
- estado (activa / inactiva)

🕒 **funciones**

Representa cada proyección de una película.

- id (PK)
- pelicula_id (FK → peliculas.id)
- fecha
- hora
- sala (opcional si solo hay una)
- precio
- estado (disponible / cancelada)

🪑 **asientos**

(Estructura fija de la sala)

- id (PK)
- numero (1 - 150)
- fila (A, B, C…)
- columna (1,2,3…)
- estado (activo/inactivo)

💡 *Tip:* Puedes precargar los 150 asientos.

🎟️ **tiquetes**

- id (PK)
- codigo (único)
- usuario_id (FK → usuarios.id, opcional)
- funcion_id (FK → funciones.id)
- fecha_compra
- total
- estado (activo / usado / cancelado)

🪑 **detalle_tiquete**

(Relación muchos a muchos entre tiquete y asientos)

- id (PK)
- tiquete_id (FK → tiquetes.id)
- asiento_id (FK → asientos.id)
- precio_unitario

🔗 **Relaciones clave**

- 1 película → muchas funciones
- 1 función → muchos tiquetes
- 1 tiquete → muchos asientos
- 1 asiento → puede estar en muchas funciones (pero no repetido por función)

⚠️ **Regla crítica del sistema**

👉 **Un asiento NO puede venderse dos veces para la misma función**

Esto se garantiza con:

- Validación en backend
- Restricción lógica:

UNIQUE(funcion_id, asiento_id)

🔄 **2. Flujo de Uso del Sistema**

Te lo explico como si fuera la experiencia real del usuario 👇

🎬 **Flujo 1: Administración (Backoffice)**

1. Admin inicia sesión
2. Registra una película
3. Programa funciones:
    - Selecciona película

    - Define fecha y hora
    - Define precio

📌 Resultado: cartelera disponible

👷 Flujo 2: Usuario compra tiquetes

1. Usuario entra al sistema
2. Visualiza cartelera
3. Selecciona una película
4. Selecciona una función

💺 Selección de asientos

5. Sistema muestra:
    - 150 sillas en formato grid
    - Estados:
        - 🟢 Disponible
        - 🔴 Ocupado
6. Usuario selecciona asientos

💳 Compra

7. Sistema calcula total:

total = cantidad_asientos * precio

8. Usuario confirma compra
9. Sistema:
    - Genera código único
    - Guarda tiquete
    - Asocia asientos

🎟️ Flujo 3: Validación (Ingreso al cine)

1. Usuario presenta código (QR o texto)
2. Sistema valida:
    - ¿Existe?
    - ¿Está activo?
    - ¿Ya fue usado?
3. Resultado:
    - ✅ Válido → marcar como “usado”
    - ❌ Inválido / usado → rechazar

📊 Flujo 4: Reportes (Admin)

1. Admin accede al dashboard
2. Visualiza:
    - Ventas por día
    - Ocupación por función
    - Películas más vistas

🧭 3. Flujo Técnico (Arquitectura)

Frontend

- Consume API REST
- Muestra:
    - Cartelera
    - Asientos
    - Compra

Backend

- Maneja lógica:
    - Validación de asientos

    - Generación de códigos
    - Control de estados

Base de datos

- Persistencia
- Integridad

🔥 **4. Endpoints clave (ejemplo)**

🎬 **Películas**

- GET /peliculas
- POST /peliculas

🕒 **Funciones**

- GET /funciones
- POST /funciones

💺 **Asientos por función**

- GET /funciones/{id}/asientos

🎟️ **Compra**

- POST /tiquetes

✅ **Validación**

- POST /tiquetes/validar

🚀 **5. Mejores prácticas (clave para tus aprendices)**

- Usar transacciones al comprar:
  👉 evita doble venta
- Validar en backend SIEMPRE
- Manejar estados correctamente
- Separar lógica (MVC o capas)